

Suraksha Shield API Documentation

Insurance Policy Optimizer

Suraksha Shield — API Documentation

Project Info

Field	Value
Project Name	Suraksha Shield — Insurance Policy Optimizer
Backend	Java 17 + Spring Boot 3.2.5
Database	PostgreSQL
Authentication	JWT (HS256, 24-hour expiry)
Base URL	<code>http://localhost:3647</code>
All API paths	Prefixed with <code>/api</code>
Content-Type	<code>application/json</code> (except CSV upload)

Roles

Role	Description
<code>Public</code>	No token needed
<code>ROLE_USER</code>	Regular registered user — needs Bearer token
<code>ROLE_ADMIN</code>	Admin — needs Bearer token from admin login

JWT Authentication

How it works:

1. Call login endpoint → receive `token` in response
2. Store token (localStorage or memory)
3. For every protected request, send header:

```
Authorization: Bearer <token>
```

- Algorithm: HS256
- Expiry: 86400000 ms = **24 hours**
- Subject claim = user's email address

Standard auth header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
```

Error Response Shape

All error responses from the global exception handler follow this structure:

```
{
  "timestamp": "2026-08-23T10:00:00.000+00:00",
  "message": "Human-readable error message",
  "details": "uri=/api/..."
}
```

Validation errors (400) return an extra `errors` object:

```
{
  "timestamp": "2026-08-23T10:00:00.000+00:00",
  "message": "Validation failed",
  "errors": {
    "email": "Email is required",
    "password": "Password is required"
  }
}
```

Group 1 — Authentication APIs

API 1 — Register User

Field	Value
Method	POST
URL	http://localhost:3647/api/auth/register
Auth Required	No
Role	Public

Request Headers:

```
Content-Type: application/json
```

Request Body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "password": "secret123"
}
```

Request Body Fields:

Field	Type	Required	Validation	Example
firstName	String	Yes	Not blank	"John"
lastName	String	Yes	Not blank	"Doe"
email	String	Yes	Valid email format	"john@example.com"
password	String	Yes	Min 6 characters	"secret123"

Success Response — HTTP 200 OK:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "type": "Bearer",
  "id": 1,
  "email": "john@example.com",
}
```

```
"role": "USER",
"firstName": "John",
"lastName": "Doe"
}
```

Success Response Fields:

Field	Type	Description
token	String	JWT token — store and use for all authenticated requests
type	String	Always <code>"Bearer"</code>
id	Long	User's database ID
email	String	Registered email
role	String	Always <code>"USER"</code> for this endpoint
firstName	String	User's first name
lastName	String	User's last name

Error Responses:

Status	When	Response body message
400 Bad Request	Email already registered	<code>"Email address already in use!"</code>
400 Bad Request	Validation failure	<code>"Validation failed" + errors object</code>

Frontend Notes:

- After successful registration the user is automatically logged in — store the token immediately, no separate login call needed.
- Redirect the user to their dashboard or policies page after saving the token.

API 2 — User Login

Field	Value
Method	POST
URL	<code>http://localhost:3647/api/auth/login</code>
Auth Required	No
Role	Public

Request Headers:

```
Content-Type: application/json
```

Request Body:

```
{
  "email": "john@example.com",
  "password": "secret123"
}
```

Request Body Fields:

Field	Type	Required	Validation	Example
email	String	Yes	Valid email format	<code>"john@example.com"</code>
password	String	Yes	Not blank	<code>"secret123"</code>

Success Response — HTTP 200 OK: (same shape as Register — see table above)

Error Responses:

Status	When	Response body message
401 Unauthorized	Wrong email or password	"Invalid email or password"
403 Forbidden	Credentials belong to an ADMIN (not a user)	"Access Denied: Invalid User Credentials"
400 Bad Request	Validation failure (empty fields)	"Validation failed"

Frontend Notes:

- Save `token`, `id`, `email`, `firstName`, `lastName`, and `role` from the response.
- This endpoint is for regular users only. Admin credentials will return 403.

API 3 — Admin Login

Field	Value
Method	POST
URL	<code>http://localhost:3647/api/auth/admin/login</code>
Auth Required	No
Role	Public

Request Headers:

```
Content-Type: application/json
```

Request Body:

```
{
  "email": "admin@gmail.com",
  "password": "admin123"
}
```

Request Body Fields:

Field	Type	Required	Validation	Example
email	String	Yes	Valid email format	<code>"admin@gmail.com"</code>
password	String	Yes	Not blank	<code>"admin123"</code>

Success Response — HTTP 200 OK:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "type": "Bearer",
  "id": 1,
  "email": "admin@gmail.com",
  "role": "ADMIN"
}
```

Response Fields:

Field	Type	Description
token	String	JWT token for admin — use in all admin API requests
type	String	Always <code>"Bearer"</code>

id	Long	Admin's database ID
email	String	Admin email
role	String	Always "ADMIN" for this endpoint

Note: `firstName` and `lastName` are NOT returned for admin — the admin entity does not have those fields.

Error Responses:

Status	When	Response body message
401 Unauthorized	Wrong email or password	"Invalid email or password"
403 Forbidden	Credentials belong to a USER (not admin)	"Access Denied: Invalid Admin Credentials"
400 Bad Request	Validation failure	"Validation failed"

Default Admin Credentials (seeded on first startup):

- Email: `admin@gmail.com`
- Password: `admin123`

Group 2 — Policy APIs

API 4 — Get Policy by ID

Field	Value
Method	GET
URL	<code>http://localhost:3647/api/policies/{id}</code>
Auth Required	No
Role	Public

Path Parameters:

Parameter	Type	Required	Description	Example
id	Long	Yes	Database ID of the policy	1

No request body.

Success Response — HTTP 200 OK:

```
{
  "id": 1,
  "name": "LIC Jeevan Suraksha",
  "type": "life",
  "premium": 5000,
  "coverage": 300000,
  "riskLevel": "low",
  "provider": "LIC"
}
```

Response Fields:

Field	Type	Description
id	Long	Policy ID

name	String	Policy name
type	String	life , health , car , home , or travel
premium	Integer	Annual premium in INR
coverage	Integer	Total coverage amount in INR
riskLevel	String	low , medium , or high
provider	String	Insurance provider name

Note: `createdBy` (admin reference) is never included in the response — it is annotated `@JsonIgnore`.

Error Responses:

Status	When	Response body message
404 Not Found	No policy with that ID	"Policy not found with id: {id}"

API 5 — Search & Optimize Policies

Field	Value
Method	GET
URL	http://localhost:3647/api/policies/results
Auth Required	No
Role	Public

Query Parameters:

Parameter	Type	Required	Description	Accepted Values
name	String	No	Search by policy name (partial match, case-insensitive)	any string
type	String	No	Filter by policy type	life , health , car , home , travel
riskLevel	String	No	Filter by risk level	low , medium , high
max_premium	Integer	No	Max annual premium budget (INR). Default: 20000	any positive int
coverage_min	Integer	No	Minimum required coverage (INR). Default: 0	any positive int
coverage_max	Integer	No	Maximum coverage cap (INR). Default: 5000000	any positive int
show_all	Boolean	No	If true , skip optimization and return all policies sorted by premium	true / false

Behaviour rules (important for frontend):

Scenario	What happens
<code>show_all=true</code>	Returns all policies sorted by premium ascending. <code>combinationMode = false</code> .
<code>name</code> provided	Bypasses optimization. Returns name-matched policies sorted

	by premium. <code>combinationMode = false</code> .
Filters only	Runs knapsack optimization. Returns cheapest combination meeting coverage. <code>combinationMode = true</code> .
No params at all	Runs optimization on full catalog with defaults (max_premium=20000, coverage range 0-5000000).

Success Response — HTTP 200 OK:

```
{
  "policies": [
    {
      "id": 3,
      "name": "Star Health Basic",
      "type": "health",
      "premium": 3000,
      "coverage": 200000,
      "riskLevel": "low",
      "provider": "Star Health"
    }
  ],
  "combinationMode": true,
  "totalPremium": 3000,
  "totalCoverage": 200000
}
```

Response Fields:

Field	Type	Description
policies	Array<Policy>	List of matched/optimized policies
combinationMode	Boolean	<code>true</code> = result is an optimized combination; <code>false</code> = simple list
totalPremium	Integer	Sum of premiums in the combination (0 when <code>combinationMode</code> is false)
totalCoverage	Integer	Sum of coverage in the combination (0 when <code>combinationMode</code> is false)

Error Responses:

Status	When
500 Internal Server Error	Unexpected server failure

Frontend Notes:

- When `combinationMode` is `true`, display `totalPremium` and `totalCoverage` as a summary banner.
- When `combinationMode` is `false` (name search or `show_all`), display as a plain list.
- An empty `policies` array with `combinationMode: true` means no valid combination was found within the given budget/coverage constraints — show a “no results” message.

API 6 — Download Policy Summary

Field	Value
Method	GET
URL	<code>http://localhost:3647/api/policies/{id}/download</code>
Auth Required	No
Role	Public

Path Parameters:

Parameter	Type	Required	Description
id	Long	Yes	Policy ID

Success Response — HTTP 200 OK:

- Content-Type: `text/plain`
- Content-Disposition: `attachment; filename="<PolicyName>_summary.txt"`
- Body: plain-text policy summary document

Error Responses:

Status	When
404 Not Found	Policy ID does not exist

Frontend Notes:

- Do not use `fetch().json()` — this returns a file, not JSON.
- Use `window.open(url)` or create a hidden `<a>` tag with `download` attribute and click it programmatically.

```
const link = document.createElement('a');
link.href = `http://localhost:3647/api/policies/${id}/download`;
link.download = '';
document.body.appendChild(link);
link.click();
document.body.removeChild(link);
```

API 7 — Allocate Policy to Profile

Field	Value
Method	<code>POST</code>
URL	<code>http://localhost:3647/api/policies/{id}/allocate</code>
Auth Required	Yes
Role	<code>ROLE_USER</code>

Request Headers:

```
Authorization: Bearer <USER_JWT_TOKEN>
```

Path Parameters:

Parameter	Type	Required	Description
id	Long	Yes	Policy ID to allocate

Success Response — HTTP 200 OK:

```
{
  "message": "Policy successfully allocated to your profile!"
}
```

Error Responses:

Status	When	Response body message
400 Bad Request	Policy already allocated	"You have already allocated this policy."
401 Unauthorized	No/invalid JWT token	—
403 Forbidden	Admin token used instead of user token	—

404 Not Found	Policy ID does not exist	"Policy not found with id: {id}"
---------------	--------------------------	----------------------------------

Frontend Notes:

- Only logged-in users can allocate. Redirect to login if no token is present.
- Disable the “Allocate” button after a successful allocation to prevent duplicate calls.

Group 3 — User APIs

API 8 — Get User Profile

Field	Value
Method	GET
URL	http://localhost:3647/api/users/profile
Auth Required	Yes
Role	ROLE_USER

Request Headers:

Authorization: Bearer <USER_JWT_TOKEN>

Success Response — HTTP 200 OK:

```
{
  "id": 1,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "allocatedPolicies": [
    {
      "id": 3,
      "name": "Star Health Basic",
      "type": "health",
      "premium": 3000,
      "coverage": 200000,
      "riskLevel": "low",
      "provider": "Star Health"
    }
  ]
}
```

Response Fields:

Field	Type	Description
id	Long	User’s database ID
firstName	String	User’s first name
lastName	String	User’s last name
email	String	User’s email
allocatedPolicies	Array<Policy>	All policies the user has allocated. Empty array if none.

Error Responses:

Status	When
401 Unauthorized	No/invalid/expired JWT

403 Forbidden	Admin token used
---------------	------------------

API 9 — Deallocate Policy from Profile

Field	Value
Method	DELETE
URL	http://localhost:3647/api/users/policies/{policyId}/deallocate
Auth Required	Yes
Role	ROLE_USER

Request Headers:

```
Authorization: Bearer <USER_JWT_TOKEN>
```

Path Parameters:

Parameter	Type	Required	Description
policyId	Long	Yes	ID of the policy to remove from profile

Success Response — HTTP 200 OK:

```
{
  "message": "Policy removed from your profile."
}
```

Error Responses:

Status	When	Response body message
400 Bad Request	Policy not in user's profile	"This policy was not allocated to your profile."
401 Unauthorized	No/invalid JWT	—
404 Not Found	Policy ID does not exist	"Policy not found with id: {id}"

Group 4 — Admin APIs

API 10 — Get All Policies (Admin Dashboard)

Field	Value
Method	GET
URL	http://localhost:3647/api/admin/dashboard
Auth Required	Yes
Role	ROLE_ADMIN

Request Headers:

```
Authorization: Bearer <ADMIN_JWT_TOKEN>
```

Query Parameters:

--	--	--	--	--

Parameter	Type	Required	Default	Description
page	Integer	No	0	Page number (zero-based)
size	Integer	No	50	Number of policies per page

Success Response — HTTP 200 OK (Spring Page object):

```
{
  "content": [
    {
      "id": 1,
      "name": "LIC Jeevan Suraksha",
      "type": "life",
      "premium": 5000,
      "coverage": 300000,
      "riskLevel": "low",
      "provider": "LIC"
    }
  ],
  "totalElements": 35,
  "totalPages": 1,
  "size": 50,
  "number": 0,
  "first": true,
  "last": true
}
```

Response Fields:

Field	Type	Description
content	Array<Policy>	Policies on the current page, sorted by id descending
totalElements	Integer	Total number of policies in the database
totalPages	Integer	Total number of pages
size	Integer	Page size requested
number	Integer	Current page number (0-based)
first	Boolean	True if this is the first page
last	Boolean	True if this is the last page

Error Responses:

Status	When
401 Unauthorized	No/invalid JWT
403 Forbidden	User token used instead of admin token

API 11 — Create Policy

Field	Value
Method	POST
URL	http://localhost:3647/api/admin/policies
Auth Required	Yes
Role	ROLE_ADMIN

Request Headers:

```
Authorization: Bearer <ADMIN_JWT_TOKEN>
Content-Type: application/json
```

Request Body:

```
{
  "name": "LIC Jeevan Suraksha",
  "type": "life",
  "premium": 5000,
  "coverage": 300000,
  "riskLevel": "low",
  "provider": "LIC"
}
```

Request Body Fields:

Field	Type	Required	Validation	Accepted Values
name	String	Yes	Not blank	Any string
type	String	Yes	Not blank	life, health, car, home, travel
premium	Integer	Yes	≥ 0	Positive integer (INR)
coverage	Integer	Yes	≥ 0	Positive integer (INR)
riskLevel	String	Yes	Not blank	low, medium, high
provider	String	Yes	Not blank	Any string

Success Response — HTTP 200 OK: Returns the created Policy object (same shape as Get Policy by ID response).

Error Responses:

Status	When
400 Bad Request	Validation failure (missing/invalid fields)
401 Unauthorized	No/invalid JWT
403 Forbidden	Non-admin token

API 12 — Update Policy

Field	Value
Method	PUT
URL	http://localhost:3647/api/admin/policies/{id}
Auth Required	Yes
Role	ROLE_ADMIN

Request Headers:

```
Authorization: Bearer <ADMIN_JWT_TOKEN>
Content-Type: application/json
```

Path Parameters:

Parameter	Type	Required	Description
id	Long	Yes	ID of the policy to update

Request Body: Same fields as Create Policy (all fields required — this is a full replacement).

Success Response — HTTP 200 OK: Returns the updated Policy object.

Error Responses:

Status	When
400 Bad Request	Validation failure
401 Unauthorized	No/invalid JWT
403 Forbidden	Non-admin token
404 Not Found	Policy ID does not exist

API 13 — Delete Policy

Field	Value
Method	DELETE
URL	http://localhost:3647/api/admin/policies/{id}
Auth Required	Yes
Role	ROLE_ADMIN

Request Headers:

```
Authorization: Bearer <ADMIN_JWT_TOKEN>
```

Path Parameters:

Parameter	Type	Required	Description
id	Long	Yes	ID of the policy to delete

Success Response — HTTP 200 OK:

```
{
  "message": "Policy deleted successfully!"
}
```

Error Responses:

Status	When
401 Unauthorized	No/invalid JWT
403 Forbidden	Non-admin token
404 Not Found	Policy ID does not exist

API 14 — Bulk Import Policies from CSV

Field	Value
Method	POST
URL	http://localhost:3647/api/admin/policies/import
Auth Required	Yes
Role	ROLE_ADMIN

Request Headers:

Authorization: Bearer <ADMIN_JWT_TOKEN>
Content-Type: multipart/form-data

Do NOT manually set Content-Type in your fetch/Axios call when sending FormData — the browser sets the boundary automatically.

Request Body: multipart/form-data

Form Field	Type	Required	Description
file	File	Yes	CSV file to import

Standard CSV required columns: name, type, premium, coverage, riskLevel, provider

Success Response — HTTP 200 OK:

```
{  
  "imported": 30,  
  "skipped": 2,  
  "errors": []  
}
```

Fields in the success response are determined by `CsvImportService` — exact shape may vary. The response is a Map object.

Error Responses:

Status	When
400 Bad Request	Invalid/malformed CSV file
401 Unauthorized	No/invalid JWT
403 Forbidden	Non-admin token

File size limit: 20 MB maximum.

JWT Authentication Flow

```
Frontend (Login Form)  
  |  
  | POST /api/auth/login { email, password }  
  v  
Backend (AuthController)  
  |  
  | Validates credentials via Spring Security  
  | Verifies ROLE_USER or ROLE_ADMIN  
  | Generates JWT (HS256, 24h expiry)  
  v  
Frontend receives:  
  { token, id, email, role, firstName, lastName }  
  |  
  | Stores token in localStorage / memory  
  v  
Every Protected Request:  
  | Header: Authorization: Bearer <token>  
  v  
JwtAuthenticationFilter  
  |  
  | Validates token signature + expiry  
  | Extracts email from token subject
```

```
| Loads user from database
v
Spring Security
|
| Checks role against endpoint requirement
v
Protected API executes and returns response
```

Complete API Quick Reference Table

Method	Endpoint	Auth	Role	Purpose
POST	/api/auth/register	No	Public	Register new user (auto-login)
POST	/api/auth/login	No	Public	User login → JWT
POST	/api/auth/admin/login	No	Public	Admin login → JWT
GET	/api/policies/results	No	Public	Search & optimize policies
GET	/api/policies/{id}	No	Public	Get single policy by ID
GET	/api/policies/{id}/download	No	Public	Download policy summary as .txt
POST	/api/policies/{id}/allocate	Yes	USER	Allocate policy to user profile
GET	/api/users/profile	Yes	USER	Get user profile + allocated policies
DELETE	/api/users/policies/{policyId}/deallocate	Yes	USER	Remove policy from user profile
GET	/api/admin/dashboard	Yes	ADMIN	Get all policies paginated
POST	/api/admin/policies	Yes	ADMIN	Create new policy
PUT	/api/admin/policies/{id}	Yes	ADMIN	Update existing policy
DELETE	/api/admin/policies/{id}	Yes	ADMIN	Delete policy
POST	/api/admin/policies/import	Yes	ADMIN	Bulk import from CSV file

Error Handling Reference

HTTP Status	Meaning	When frontend receives it	What frontend should do
200 OK	Success	Request completed successfully	Read and display the response
400 Bad Request	Invalid input or business rule violation	Missing fields, failed validation, duplicate email, policy already allocated	Show the <code>message</code> or <code>errors</code> fields to the user
401 Unauthorized	Authentication failed	Wrong credentials, missing token, expired token	Redirect to login page, clear stored token
403 Forbidden	Access denied	Using a user token on admin route (or vice versa), role	Show “Access denied” message, redirect

		mismatch	appropriately
404 Not Found	Resource does not exist	Invalid policy ID, user not found	Show “Not found” message
500 Internal Server Error	Unexpected server error	Server crash, DB connection failure	Show generic “Something went wrong” message, do not expose details

Frontend Integration Examples

Register

```
const response = await fetch("http://localhost:3647/api/auth/register", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    firstName: "John",
    lastName: "Doe",
    email: "john@example.com",
    password: "secret123"
  })
});
const data = await response.json();
if (response.ok) {
  localStorage.setItem("token", data.token);
  localStorage.setItem("role", data.role);
}
```

Login

```
const response = await fetch("http://localhost:3647/api/auth/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email: "john@example.com", password: "secret123" })
});
const data = await response.json();
if (response.ok) {
  localStorage.setItem("token", data.token);
}
```

Get Optimized Policies

```
const response = await fetch(
  "http://localhost:3647/api/policies/results?type=health&riskLevel=low&max_premium=10000&coverage_min=200000"
);
const data = await response.json();
// data.policies -> array of policies
// data.combinationMode -> true/false
// data.totalPremium, data.totalCoverage -> summary
```

Allocate Policy (authenticated)

```
const token = localStorage.getItem("token");
const response = await fetch(`http://localhost:3647/api/policies/${policyId}/allocate`, {
  method: "POST",
  headers: { "Authorization": `Bearer ${token}` }
});
const data = await response.json();
```

Get User Profile (authenticated)


```
const token = localStorage.getItem("token");
const response = await fetch("http://localhost:3647/api/users/profile", {
  headers: { "Authorization": `Bearer ${token}` }
});
const data = await response.json();
// data.allocatedPolicies -> array of the user's policies
```

Create Policy — Admin (Axios)

```
import axios from "axios";
const adminToken = localStorage.getItem("adminToken");
const response = await axios.post(
  "http://localhost:3647/api/admin/policies",
  {
    name: "LIC Jeevan Suraksha",
    type: "life",
    premium: 5000,
    coverage: 300000,
    risklevel: "low",
    provider: "LIC"
  },
  { headers: { Authorization: `Bearer ${adminToken}` } }
);
```

CSV Import — Admin (multipart)

```
const adminToken = localStorage.getItem("adminToken");
const formData = new FormData();
formData.append("file", csvFile); // csvFile is a File object from <input type="file">
const response = await fetch("http://localhost:3647/api/admin/policies/import", {
  method: "POST",
  headers: { "Authorization": `Bearer ${adminToken}` },
  // DO NOT set Content-Type manually - browser sets it with boundary
  body: formData
});
```

Download Policy Summary

```
const link = document.createElement("a");
link.href = `http://localhost:3647/api/policies/${policyId}/download`;
link.download = "";
document.body.appendChild(link);
link.click();
document.body.removeChild(link);
```